

--- title: "結合無監督動態分群與深度強化學習之配對交易系統建構與實證：以 S&P 500 為例" format: revealjs: scrollable: true smaller: true slide-number: true slide-level: 3 execute: # <--- 新增這層控制執行行為 echo: true # <--- 強制整份簡報預設都要印出程式碼 output-location: default ---

1. 研究動機 (Research Motivation)

建立此基準模型 (Baseline Model) 的核心目的，是為了提供客觀、可靠的對比標竿。

在導入複雜的深度增強式學習 (Deep Reinforcement Learning, DRL) 之前，必須先建立穩固的基礎：

- 確保資料純淨度：**妥善處理歷史數據中的生存者偏誤 (Survivorship Bias)，確保回測結果貼近真實市場。
- 建立傳統交易邏輯底盤：**實作具備共整合 (Cointegration) 與 Z-Score 邏輯的傳統配對交易系統，驗證均值回歸特性的有效性，作為後續 RL 代理人學習的基礎狀態與邏輯參考。

2. 文獻回顧 (Literature Review)

配對交易 (Pairs Trading) 為統計套利 (Statistical Arbitrage) 中經典的市場中性策略，本階段實作主要基於以下經典框架與最新研究進行建構：

- 傳統配對交易框架：**參考 Gatev, Goetzmann & Rouwenhorst (2006) 提出的經典方法論。該文獻奠定了使用距離法 (Sum of Squared Differences, SSD) 來篩選潛在配對，並採用滾動視窗 (Rolling Window) 機制的業界標準。
- 現代特徵擴展與機器學習應用：**參考 Yufei Sun (2025) 對於配對交易系統性文獻的最新整理。其研究確立了將總體經濟指標 (Macroeconomic Indicators，例如 VIX) 納入考量，以及後續應用機器學習進行動態分群 (Clustering) 的發展方向，這也成為我們後續強化學習模型的特徵工程藍圖。

3. 資料說明與前置處理 (Data Description & Preprocessing)

- 資料來源：**本研究使用 S&P 500 歷史日 K 線資料庫 (`sp500_data.db`)，資料期間涵蓋 2000-01-01 至 2025-12-31 的數據。
- 資料清洗步驟：**
 - 前向填充與剔除稀疏股票：**針對價格序列進行前向填充 (Forward-fill) 最多 5 天，避免假日或停牌影響，並剔除有效交易日過少的稀疏股票。
 - 生存者偏誤 (Survivorship Bias) 處理：**確保資料庫中包含歷史已下市成分股之數據，避免回測時因只考慮現有成分股而產生高估績效的生存者偏誤。
 - 宏觀指標融合：**自動整合 Yahoo Finance 的 VIX 波動率指數至特徵矩陣中，為未來環境狀態空間 (State Space) 提供總體經濟環境的降維特徵。

4. 策略說明與實作細節 (Strategy Description & Implementation Details)

策略依據：Gatev, Goetzmann & Rouwenhorst (2006)

策略總覽

配對交易為市場中性的統計套利策略。

階段	名稱	說明
1	初始化與資料收集	參數、SQLite、清理
2	配對選擇	協整檢驗 → SSD 排序
3	策略執行	Z-Score 訊號 → PnL
4	績效評估	Sharpe、MDD、CAGR

階段一：初始化與資料收集

1-1 套件安裝

```
In [1]: import subprocess, sys
for pkg in ['statsmodels','plotly','ipywidgets','kaleido','scikit-learn','yfinan
subprocess.check_call([sys.executable, '-m', 'pip', 'install', pkg, '-q'])
import warnings; warnings.filterwarnings('ignore')
import sqlite3, numpy as np, pandas as pd
import logging
from itertools import combinations
from statsmodels.tsa.stattools import coint
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.express as px
import ipywidgets as widgets
from IPython.display import display, HTML
pd.set_option('display.float_format', '{:.4f}'.format)
import yfinance as yf
import time
import os
# print('套件導入完成。')
```

1-2 全域參數設定

使用者只需修改此區塊，即可調整所有策略參數。

參數	預設值	說明
DB_PATH	sp500_data.db	SQLite 資料庫路徑
FORMATION_WINDOW	252	形成期 (12M)
TRADING_WINDOW	126	交易期 (6M)
ROLLING_WINDOW	20	滾動窗口 (1M)

參數	預設值	說明
COINT_P_VALUE	0.01	協整檢驗顯著水準
TOP_N_PAIRS	[5, 20]	Top-N 配對數
Z_ENTRY	2	進場閾值
Z_EXIT	0	出場閾值
MAX_LOSS_PCT	3.0	最大停損百分比
TRANSACTION_COST	0.0029	單邊成本 0.1%
INITIAL_CAPITAL	7000	初始資金7000美金
CAPITAL_TRANCHES	7	分割成七等分

```
In [2]: import os as _os

# =====
# 🚦 全局模式切換開關 (True: 快速開發測試 / False: 論文最終完整回測)
# =====
FAST_TEST_MODE = True

if FAST_TEST_MODE:
    print("🚀 啟動【快速測試模式】: 僅回測 2019-2022 年間之科技板塊, 預計數分鐘內完成
    # 縮短時間: 涵蓋 2020 疫情崩盤與 2022 升息的壓力測試區間
    START_DATE = '2019-01-01'
    END_DATE = '2022-12-31'
    # 縮小股票池: 僅測試單一板塊, 運算量大幅減少
    TARGET_SECTOR = 'Information Technology'
else:
    print("🐢 啟動【完整回測模式】: 回測 2000-2025 年間之全市場 S&P 500, 需較長運算時間
    # 論文要求的全樣本時間
    START_DATE = '2000-01-01'
    END_DATE = '2025-12-31'
    # 測試全市場 (設為 None 代表不限制單一產業)
    TARGET_SECTOR = None

# =====
# 📊 共用策略參數 (兩種模式皆適用)
# =====
DB_PATH = r'..\data\sp500_data.db'
# 產業動態補齊開關與快取路徑
USE_DYNAMIC_SECTORS = True
IMPUTED_SECTOR_PATH = r'..\data\imputed_sectors.csv'

# 視窗滾動參數
FORMATION_WINDOW = 252 # 形成期 (約一年)
TRADING_WINDOW = 126 # 交易期 (約半年)
ROLLING_WINDOW = 20 # 滾動步長 (約一個月)
MIN_HISTORY_DAYS = 200

# 配對與統計檢定參數
TOP_N_PAIRS = [1, 3] # 觀察 Top-5 與 Top-20 的差異
COINT_P_VALUE = 0.01 # 嚴格的共整合 p-value 門檻
SECTOR_NEUTRAL = True # 是否限制同產業配對 (當 TARGET_SECTOR 為 None 時生效)

# 交易執行與資金控管參數
Z_ENTRY = 2 # 進場門檻
```

```

Z_EXIT = 0.0          # 出場門檻 (均值回歸)
MAX_LOSS_PCT = 3       # 嚴格的單筆未實現損益停損 (5%)
TRANSACTION_COST = 0.0029 # 雙邊交易手續費 (0.29%)
HEDGE_RATIO = 1.0      # 避險比例 (後續已被動態 Beta 取代, 此處保留作為備用)

INITIAL_CAPITAL = 7000 # 初始本金
CAPITAL_TRANCHES = 7   # 滾動視窗資金切割份數

# 確保快取檔案的上一層資料夾存在
os.makedirs(os.path.dirname(IMPURED_SECTOR_PATH), exist_ok=True)

# 自動轉換單一數值為 List 以防止後續 Iterable 報錯
if isinstance(TOP_N_PAIRS, int):
    TOP_N_PAIRS = [TOP_N_PAIRS]

```

 啟動【快速測試模式】：僅回測 2019-2022 年間之科技板塊，預計數分鐘內完成。

1-3 資料讀取與清理

清理流程: 前向填充(最多5天) → 剪除稀疏股票 → 對齊日期

資料庫結構 (sp500_data.db):

資料表	關鍵欄位
daily_prices	ticker, date, adj_close
tickers	ticker, sector

```

In [3]: def load_data_from_db(db_path, start_date, end_date):
        """Load price and sector data from SQLite."""
        conn = sqlite3.connect(db_path)
        price_queries = [
            (f"SELECT date, ticker, open, high, low, adj_close AS close, volume FROM
             f"WHERE date BETWEEN '{start_date}' AND '{end_date}' ORDER BY date"),
            (f"SELECT date, ticker, open, high, low, close, volume FROM stock_prices
             f"WHERE date BETWEEN '{start_date}' AND '{end_date}' ORDER BY date"),
        ]
        prices_df = None
        for q in price_queries:
            try:
                prices_df = pd.read_sql_query(q, conn, parse_dates=['date'])
                if len(prices_df) > 0:
                    print(f'Price rows: {len(prices_df):,}')
                    break
            except Exception:
                continue
        if prices_df is None or len(prices_df) == 0:
            tbls = pd.read_sql_query("SELECT name FROM sqlite_master WHERE type='tab
            conn.close()
            raise RuntimeError(f"No price table. Tables: {tbls['name'].tolist()}")
        sector_queries = [
            "SELECT ticker, sector FROM tickers",
            "SELECT ticker, sector FROM sp500_components GROUP BY ticker",
        ]
        sector_df = None
        for q in sector_queries:
            try:
                sector_df = pd.read_sql_query(q, conn)

```

```

        if len(sector_df) > 0:
            print(f'Sector rows: {len(sector_df):,}')
            break
    except Exception:
        continue
conn.close()
if sector_df is None or len(sector_df) == 0:
    print('WARNING: No sector table, using Unknown.')
    sector_df = pd.DataFrame({'ticker': prices_df['ticker'].unique(), 'sector': 'Unknown'})

# [Mod] Mitigate Survivorship Bias: Check if delisted components exist
max_date = prices_df['date'].max()
max_dates = prices_df.groupby('ticker')['date'].max()
delisted_count = (max_dates < max_date - pd.Timedelta(days=30)).sum()
if delisted_count < 10:
    import logging
    logging.warning("STRICT RESEARCH LIMITATION: Database lacks historically")

return prices_df, sector_df

def fix_unknown_sectors(sector_df, use_dynamic=True, save_path=r'data\imputed_sectors.csv'):
    """具備本機快取與全域開關控制的產業補齊模組"""
    # 1. 開關判斷：如果不使用動態補齊，直接原封不動回傳
    if not use_dynamic:
        print("不使用動態產業補齊，維持原始 Unknown 分類作為對照組。")
        return sector_df

    # 確保儲存的目錄 (data\) 存在
    os.makedirs(os.path.dirname(save_path), exist_ok=True)

    # 2. 快取讀取：如果已經抓過並存檔，直接載入
    if os.path.exists(save_path):
        print(f"從本機快取載入已補齊的產業分類: {save_path}")
        cached_df = pd.read_csv(save_path)
        update_df = cached_df.set_index('ticker')
        sector_df = sector_df.set_index('ticker')
        sector_df.update(update_df)
        return sector_df.reset_index()

    # 3. API 抓取：如果沒有快取，執行連線作業
    unknown_mask = sector_df['sector'] == 'Unknown'
    unknown_tickers = sector_df[unknown_mask]['ticker'].tolist()

    if not unknown_tickers:
        return sector_df

    print(f"找不到本機快取，正在透過 API 補齊 {len(unknown_tickers)} 檔 Unknown 股票")

    yf_logger = logging.getLogger('yfinance')
    original_level = yf_logger.level
    yf_logger.setLevel(logging.CRITICAL)

    fixed_sectors = []

    for i, ticker in enumerate(unknown_tickers):
        try:
            info = yf.Ticker(ticker).info
            sector = info.get('sector', 'Unknown')
            fixed_sectors.append((ticker, sector))
        except Exception as e:
            yf_logger.error(f"Error fetching sector for {ticker}: {e}")

    # 將補齊的數據寫回 CSV
    df_to_save = pd.DataFrame(fixed_sectors, columns=['ticker', 'sector'])
    df_to_save.to_csv(save_path, index=False)

    return sector_df

```

```

        fixed_sectors.append({'ticker': ticker, 'sector': sector})
        time.sleep(0.02)
    except Exception:
        fixed_sectors.append({'ticker': ticker, 'sector': 'Unknown'})

    if (i + 1) % 50 == 0:
        print(f"已處理 {i + 1} / {len(unknown_tickers)}...")

yf_logger.setLevel(original_level)

# 4. 儲存快取：將剛抓下來的資料存成 CSV，下次就不用再抓了
fetched_df = pd.DataFrame(fixed_sectors)
fetched_df.to_csv(save_path, index=False)
print(f"API 抓取完畢！已將動態產業分類永久儲存至：{save_path}")

# 更新回原本的 DataFrame
update_df = fetched_df.set_index('ticker')
sector_df = sector_df.set_index('ticker')
sector_df.update(update_df)
sector_df = sector_df.reset_index()

remaining = len(sector_df[sector_df['sector'] == 'Unknown'])
print(f"補齊完成！剩餘真實無法識別(已下市)的 Unknown 股票數量：{remaining}")

return sector_df

def preprocess_prices(prices_df, min_days=MIN_HISTORY_DAYS):
    """Pivot, forward-fill, drop sparse tickers."""
    pivot = prices_df.pivot_table(index='date', columns='ticker', values='close')
    pivot.index = pd.to_datetime(pivot.index)
    pivot.sort_index(inplace=True)
    pivot.ffmpeg(limit=5, inplace=True)
    valid = pivot.columns[pivot.notna().sum() >= min_days]
    pivot = pivot[valid]
    print(f'Matrix: {len(pivot)} days x {len(pivot.columns)} tickers')

    # [Mod] Integrate Macroeconomic Indicator (VIX)
    import yfinance as yf
    print("Fetching VIX data...")
    try:
        vix_data = yf.download("^VIX", start=pivot.index.min(), end=pivot.index.max())
        if isinstance(vix_data.columns, pd.MultiIndex):
            vix_close = vix_data['Close'].squeeze()
        else:
            vix_close = vix_data['Close']
        vix_df = pd.DataFrame({'VIX': vix_close})
        vix_df.index = pd.to_datetime(vix_df.index).tz_localize(None)
        vix_aligned = vix_df.reindex(pivot.index).ffill()
        print("VIX feature matrix aligned.")
    except Exception as e:
        print(f"Error fetching VIX: {e}")
        vix_aligned = pd.DataFrame(index=pivot.index, columns=['VIX'])

    return pivot, vix_aligned

```

執行資料載入與特徵前置處理

本區塊將實際調用前方定義的模組，自 SQLite 提取百萬筆日 K 線與產業關聯資料，並將缺漏的 Unknown 產業透過 API 自動填補。接著把資料對齊樞紐化 (Pivot) 並整合總體經濟 VIX 波動率指標，建立乾淨的歷史面板資料 (Panel Data)。

```
In [4]: # 1. 從資料庫載入原始資料
prices_raw, sector_info = load_data_from_db(DB_PATH, START_DATE, END_DATE)

# 2. 攔截並補齊 Unknown 產業分類 (傳入全域開關與路徑)
sector_info = fix_unknown_sectors(
    sector_info,
    use_dynamic=USE_DYNAMIC_SECTORS,
    save_path=IMPUTED_SECTOR_PATH
)

# 3. 執行原有的價格矩陣轉換與 VIX 融合
price_pivot, vix_features = preprocess_prices(prices_raw)
sector_map = sector_info.set_index('ticker')['sector'].to_dict()

# 4. 顯示結果
print(pd.Series(sector_map).value_counts().head(10))
```

Price rows: 616,467

Sector rows: 843

從本機快取載入已補齊的產業分類: ..\data\imputed_sectors.csv

Matrix: 1008 days x 623 tickers

Fetching VIX data...

VIX feature matrix aligned.

Unknown	214
Industrials	94
Financials	76
Information Technology	71
Health Care	60
Consumer Discretionary	48
Consumer Cyclical	37
Real Estate	36
Consumer Staples	36
Energy	35

Name: count, dtype: int64

階段一之番外篇：特徵工程 (Feature Engineering)

本區塊將針對原始的 OHLCV 歷史數據進一步提煉出 機器學習與強化學習 適合的微觀/總體特徵。包含：

1. 計算基礎特徵 (Log_Ret、ATR、PCA 殘差、動態市場關聯)。
2. 衡量均值回歸的進階微觀特徵 (Hurst Exponent、Rolling Half-life、ADF Test、Volume Imbalance)。
3. 融合總體經濟指標 (VIX)。
4. 執行特徵標準化 (Standard Scaler)。

```
In [5]: from joblib import Parallel, delayed
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from statsmodels.tsa.stattools import adfuller
import statsmodels.api as sm
```

```

def extract_micro_ts_features(ts):
    """提取微觀特徵，並確保回傳值數量與 DataFrame 欄位一致"""
    ticker = ts.name
    try:
        ts_vals = ts.dropna()
        if len(ts_vals) < 30:
            return tuple([ticker] + [np.nan] * 7) # 回傳 8 個元素

        # 1. 最近期對數報酬率
        log_ret = np.log(ts_vals / ts_vals.shift(1)).dropna()
        log_ret_latest = log_ret.iloc[-1]

        # 2. 波動率 (近似 ATR 概念: High-Low 或純粹標準差)
        atr_latest = ts_vals.rolling(14).std().iloc[-1]

        # 3. Z-Score (相對於 60 天均值)
        roll_mean = ts_vals.rolling(60).mean()
        roll_std = ts_vals.rolling(60).std()
        z_score_latest = ((ts_vals - roll_mean) / roll_std).iloc[-1]

        # 4. 成交量失衡 (假設目前無 Volume 欄位，先填 0 或結合 VIX)
        vol_imb_latest = 0.0

        # 5. Hurst Exponent
        hurst = compute_hurst(ts_vals.values)

        # 6. Half-Life (均值回歸半衰期)
        half_life = compute_half_life(ts_vals.values)

        # 7. ADF 檢定統計量
        try:
            adf_res = adfuller(ts_vals.values, maxlag=1, regression='c', autolag='AIC')
            adf_stat = adf_res[0]
        except:
            adf_stat = np.nan

        # [修復點] 完整回傳 8 個變數，對應特徵矩陣的欄位
        return ticker, log_ret_latest, atr_latest, z_score_latest, vol_imb_latest, hurst, half_life, adf_stat

    except Exception as e:
        # 發生錯誤時也必須回傳 8 個元素，避免 unpack 失敗
        return tuple([ticker] + [np.nan] * 7)

def feature_engineering_pipeline(panel_df, vix_series, n_jobs=-1, n_pca_components=10):
    """
    建構橫斷面特徵矩陣，直接準備好輸出給 DBSCAN 使用
    panel_df: 含有 open, high, low, close, volume 的 DataFrame
    vix_series: 此觀測窗口的 VIX 時間序列
    """
    print("啟動特徵工程 Pipeline...")

    # 確認 panel_df 是以 [date, ticker] 或是可以直接 pivot
    if not isinstance(panel_df.index, pd.MultiIndex):
        panel_df = panel_df.set_index(['date', 'ticker'])

    close_pivot = panel_df['close'].unstack(level='ticker')
    log_returns = np.log(close_pivot / close_pivot.shift(1)).fillna(0)

    # A. 將 Close 價格拉平計算全市場因子 (PCA) 與 相關性
    pca = PCA(n_components=n_pca_components)

```



```

market_factors = pca.fit_transform(log_returns)
reconstructed = pca.inverse_transform(market_factors)

# 提取特有殘差 (PCA Residuals)
pca_residuals = log_returns - reconstructed
pca_res_latest = pca_residuals.iloc[-1]

# 動態 Pearson Correlation (與 PCA 第一主成分計算短週期的共同波動性)
mf_1_series = pd.Series(market_factors[:, 0], index=log_returns.index)
market_corr = log_returns.apply(lambda x: x.iloc[-20:].corr(mf_1_series.iloc[-20:]))

# B. 分配平行運算：迴圈切割給 CPU 計算各檔股票的微觀高強度特徵
tickers = close_pivot.columns
print(f"啟動多執行緒處理 {len(tickers)} 檔股票之時間序列與 ADF 微觀特徵...")

# 將每個 ticker 的歷史 df 提早整理出來給 parallel
# 避免在 function 裡面重新 filter 導致效率極差
ticker_dfs = {t: panel_df.xs(t, level='ticker') for t in tickers if t in panel_df.index}
valid_tickers = list(ticker_dfs.keys())

results = Parallel(n_jobs=n_jobs)(
    delayed(extract_micro_ts_features)(t, ticker_dfs[t]) for t in valid_tickers
)

cols = ['ticker', 'Log_Ret', 'ATR', 'Z_Score', 'Vol_Imbalance', 'Hurst', 'H_ADF']
features_df = pd.DataFrame([r for r in results if len(r) == 8], columns=cols)
features_df.columns = cols[1:]

# C. 合併市場與總體經濟層面的特徵 (Macro Awareness)
features_df['PCA_Res'] = pca_res_latest
features_df['Market_Corr'] = market_corr
features_df['VIX'] = vix_series.iloc[-1] if not vix_series.empty else np.nan

features_df = features_df.dropna() # 清理計算中斷不穩定的死點

# D. 嚴格的特徵標準化機制 (Standardization / Z-Score Scaling)
scaler = StandardScaler()
feature_columns = ['Log_Ret', 'ATR', 'Z_Score', 'Vol_Imbalance', 'Hurst', 'H_ADF']

# 過濾出成功計算出特徵的欄位，防止錯誤
valid_cols = [c for c in feature_columns if c in features_df.columns]
std_matrix = scaler.fit_transform(features_df[valid_cols])

final_standardized_df = pd.DataFrame(std_matrix, index=features_df.index, columns=valid_cols)
print(f"特徵矩陣建置完成！維度：{final_standardized_df.shape}")

return final_standardized_df, scaler

```

執行測試：萃取最後 252 天的形成期作為範例

```

In [6]: ## 取 prices_raw 最後 252 天作為形成期的範例
## 注意：在真實滾動迴圈中，panel 會是每個迴圈所取出的窗口 window
test_dates = prices_raw['date'].drop_duplicates().sort_values().tail(252)
test_panel = prices_raw[prices_raw['date'].isin(test_dates)].copy()

## 同步截取 VIX 資料
test_vix = vix_features[vix_features.index.isin(test_dates)]['VIX']

## 執行特徵工程

```

```
# std_features_matrix, feature_scaler = feature_engineering_pipeline(
#     panel_df=test_panel,
#     vix_series=test_vix,
#     n_jobs=-1
# )

# # 顯示前5筆結果 (DBSCAN / RL 的輸入狀態)
# display(std_features_matrix.head())
```

階段二：配對選擇（形成期）

Engle-Granger 協整檢驗：

$$P_{A,t} = \alpha + \beta P_{B,t} + \text{varepsilon}_t$$

如果殘差

varepsilon_t

的 ADF $p < 0.01$ ，則判定協整。

SSD 排序：

$\text{textSSD}(A, B) =$

$\text{sum}(r_A^{\text{norm}} - r_B^{\text{norm}})^2$

SSD 越小 → 趨勢越相似 → 優先選入交易期。

選定流程：同產業候選 → 協整檢驗 ($p < 0.01$) → SSD 由小到大排序 → Top-N

```
In [7]: from joblib import Parallel, delayed

def normalize_prices(prices):
    return prices / prices.iloc[0]

def compute_ssd(na, nb):
    return ((na - nb)**2).sum()

# --- 獨立的單一配對檢驗邏輯（用於多核心平行運算） ---
def test_single_coint(a, b, sector, price_window, coint_pval, min_len):
    """實作雙向 Engle-Granger 協整檢驗，並計算 OLS 共整避險比率 Beta"""
    try:
        sa = price_window[a].dropna()
        sb = price_window[b].dropna()
        idx = sa.index.intersection(sb.index)
        if len(idx) < min_len:
            return None

        # A 對 B 進行 OLS 迴歸 (A = alpha + beta * B)
        X_b = sm.add_constant(sb[idx].values)
        model_ab = sm.OLS(sa[idx].values, X_b).fit()
        beta_ab = model_ab.params[1]
        pval_ab = coint(sa[idx].values, sb[idx].values)[1]

        # B 對 A 進行 OLS 迴歸 (B = alpha + beta * A)
```

```

X_a = sm.add_constant(sa[idx].values)
model_ba = sm.OLS(sb[idx].values, X_a).fit()
beta_ba = model_ba.params[1]
pval_ba = coint(sb[idx].values, sa[idx].values)[1]

# 雙向檢驗：取 p-value 較小且具備正向經濟關聯 (beta > 0) 的方向
if pval_ab < pval_ba and pval_ab < coint_pval and beta_ab > 0:
    return {'stock_a': a, 'stock_b': b, 'sector': sector, 'coint_p': pva
elif pval_ba < coint_pval and beta_ba > 0:
    # 若 B 對 A 較佳，反轉名稱，讓系統後續統一處理 (stock_a = beta * stock_b)
    return {'stock_a': b, 'stock_b': a, 'sector': sector, 'coint_p': pva

except Exception:
    return None
return None

# --- 主篩選函數 ---
def select_pairs_for_window(price_window, sector_map, top_ns,
                             coint_pval=COINT_P_VALUE,
                             sector_neutral=SECTOR_NEUTRAL,
                             target_sector=TARGET_SECTOR):
    """嚴格遵守文獻：先 Cointegration 檢驗 -> 後 SSD 排序，並徹底剔除 Unknown 產業"""

    norm = normalize_prices(price_window.dropna(axis=1))
    valid = norm.columns.tolist()

    # 在任何分群發生之前，直接將 Unknown 股票從有效候選池中剔除
    valid = [t for t in valid if sector_map.get(t, 'Unknown') != 'Unknown']

    # 產業過濾與分群邏輯
    if target_sector is not None and target_sector != 'All':
        # 如果指定了單一產業 (例如 'Information Technology')
        valid = [t for t in valid if sector_map.get(t, 'Unknown') == target_sector]
        candidates = [(a, b, target_sector) for a, b in combinations(valid, 2)]

    elif sector_neutral:
        # 產業中性：只有同產業的股票才能互相配對
        groups = {}
        for t in valid:
            sec = sector_map.get(t, 'Unknown')
            groups.setdefault(sec, []).append(t)

        candidates = [(a, b, s) for s, ms in groups.items() if len(ms) >= 2 for
    else:
        # 全局對比 (Cross-Sector)：此時 Unknown 也已被提前剔除，不會干擾跨產業配對
        candidates = [(a, b, 'All') for a, b in combinations(valid, 2)]

    if not candidates:
        return {n: [] for n in top_ns}

    min_len = FORMATION_WINDOW * 0.8

    # 使用 JobLib 啟動多核心平行運算進行雙向協整檢驗
    coint_results = Parallel(n_jobs=-1, batch_size='auto')(
        delayed(test_single_coint)(a, b, sector, price_window, coint_pval, min_len)
        for a, b, sector in candidates
    )

    # 過濾掉未通過檢驗或回傳 None 的結果

```

```
passed = [res for res in coint_results if res is not None]

if not passed:
    return {n: [] for n in top_ns}

# 對「通過協整檢驗」的配對計算 SSD
for p in passed:
    p['ssd'] = compute_ssd(norm[p['stock_a']], norm[p['stock_b']])

# 依照 SSD 排序並選出 Top-N
df = pd.DataFrame(passed).sort_values('ssd')
return {n: df.head(n).to_dict('records') for n in top_ns}
```

階段三：策略執行（交易期）

$$Z_t = \frac{textspread_t - \mu_{hist}}{\sigma_{hist}}$$

訊號	操作
$Z > +1.5$	放空A、做多B
$Z < -1.5$	做多A、放空B
$Z(\text{絕對值}) < 0.75$	平倉
$Z(\text{絕對值}) > 3.0$	停損平倉

跨日執行法 (Wait-One-Day): 訊號後隔日執行，避免 bid-ask bounce。Dollar-Neutral: 每配對 1 做多+1 放空。

```
In [8]: def execute_pair_trades(trade_prices, pair, form_prices,
                                z_entry=Z_ENTRY, z_exit=Z_EXIT,
                                max_loss_pct=MAX_LOSS_PCT, cost=TRANSACTION_COST):
    """結合共整避險比率 (Beta) 的交易執行模組"""
```

```

a, b = pair['stock_a'], pair['stock_b']
# [修正點] 提取共整避險比率，若無則預設為 1.0
beta = pair.get('beta', 1.0)

try:
    pa_t, pb_t = trade_prices[a].dropna(), trade_prices[b].dropna()
    pa_f, pb_f = form_prices[a].dropna(), form_prices[b].dropna()
except KeyError:
    return pd.Series(0.0, index=trade_prices.index)

common = pa_t.index.intersection(pb_t.index)
fi = pa_f.index.intersection(pb_f.index)
if len(common) < 5 or len(fi) < 5:
    return pd.Series(0.0, index=trade_prices.index)

# 正規化基準點 (形成期第一天)
base_a, base_b = pa_f[fi].iloc[0], pb_f[fi].iloc[0]

# [修正點] 形成期的共整價差序列與統計特徵
fs = (pa_f[fi] / base_a) - beta * (pb_f[fi] / base_b)
mu, sd = fs.mean(), fs.std()

if sd < 1e-8: return pd.Series(0.0, index=trade_prices.index)

# 交易期正規化與 z 值計算
na = pa_t[common] / base_a
nb = pb_t[common] / base_b
z = ((na - beta * nb) - mu) / sd

z_vals, na_vals, nb_vals = z.values, na.values, nb.values
actual_stop_loss = -abs(max_loss_pct) / 100.0 if max_loss_pct > 0 else max_l

pos = 0
target_pos = 0
pna, pnb = na_vals[0], nb_vals[0]
entry_pa, entry_pb = 0.0, 0.0
in_cooldown = False
pnl_d = {}

for i in range(len(common)):
    zi, cna, cnb = z_vals[i], na_vals[i], nb_vals[i]
    dt = common[i]
    p = 0.0

    # 1. 計算「持倉」當日報酬 (注意: B 股票的權重加入了 Beta)
    if i > 0 and pos != 0:
        p += pos * ((cna - pna)/pna - beta * (cnb - pnb)/pnb)

    # 2. 跨日執行訊號 (扣除手續費)
    if target_pos != pos:
        # 總交易成本需考慮雙邊部位與 Beta 權重
        p -= 2 * cost * (1 + beta)
        if target_pos != 0:
            entry_pa, entry_pb = cna, cnb
        else:
            entry_pa, entry_pb = 0.0, 0.0
        pos = target_pos

    # 3. 盤中未實現損益 (用於停損)
    unrealized_pnl = 0.0

```

```

if pos != 0 and entry_pa > 0 and entry_pb > 0:
    unrealized_pnl = pos * ((cna - entry_pa)/entry_pa - beta * (cnb - en

# 4. 狀態機與進出場訊號
if in_cooldown:
    if abs(zi) < z_exit:
        in_cooldown = False
elif pos == 0:
    if zi > z_entry:
        target_pos = -1 # 做空價差 (空 A · 買 B)
    elif zi < -z_entry:
        target_pos = +1 # 做多價差 (買 A · 空 B)
else:
    if unrealized_pnl <= actual_stop_loss:
        target_pos = 0
        in_cooldown = True
    elif abs(zi) < z_exit:
        target_pos = 0

pna, pnb = cna, cnb
pnl_d[dt] = p

return pd.Series(pnl_d, dtype=float).reindex(trade_prices.index, fill_value=

```

階段三-1：配置配對期演算法 (HDBSCAN 動態群集與雙層過濾)

```

In [9]: from sklearn.cluster import HDBSCAN
from sklearn.metrics import silhouette_score, calinski_harabasz_score

def dynamic_hdbscan_clustering(features_df, sector_map, min_cluster_size=3):
    """
    HDBSCAN 動態分群 (板塊內層過濾)
    自動標記離群值 (Label = -1) 並輸出品質驗證指標。
    """
    features_df = features_df.copy()
    features_df['sector'] = features_df.index.map(lambda x: sector_map.get(x, 'U'))
    feature_cols = [c for c in features_df.columns if c != 'sector']

    cluster_labels = pd.Series(index=features_df.index, dtype=int)
    cluster_labels[:] = -1

    global_offset = 0
    all_metrics = []

    for sector, group in features_df.groupby('sector'):
        if len(group) < min_cluster_size * 2:
            continue

        X = group[feature_cols].values

        # 實作: HDBSCAN 演算法
        clusterer = HDBSCAN(min_cluster_size=min_cluster_size, metric='euclidean')
        labels = clusterer.fit_predict(X)

        # 自動忽略 RuntimeWarning 發生的輪廓計算
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")

```

```

        valid_mask = labels != -1
        if len(set(labels[valid_mask])) > 1:
            try:
                sil = silhouette_score(X[valid_mask], labels[valid_mask])
                ch = calinski_harabasz_score(X[valid_mask], labels[valid_mask])
                all_metrics.append({'sector': sector, 'silhouette': sil, 'calinski_harabasz': ch})
            except:
                pass

        new_labels = []
        for l in labels:
            if l == -1:
                new_labels.append(-1)
            else:
                new_labels.append(l + global_offset)

        cluster_labels.loc[group.index] = new_labels
        if len(set(labels)) > 1:
            global_offset += max(labels) + 1

    if all_metrics:
        avg_sil = np.mean([x['silhouette'] for x in all_metrics])
        avg_ch = np.mean([x['calinski_harabasz'] for x in all_metrics])
        # print(f" [Cluster Quality] Avg Silhouette: {avg_sil:.3f}, Avg CH Index: {avg_ch:.3f}")

    return cluster_labels

def select_pairs_with_hdbscan(price_window, features_df, sector_map, top_ns, coint_pval, min_len):
    if isinstance(top_ns, int):
        top_ns = [top_ns]
    """
    雙層過濾機制：
    第一層 HDBSCAN 動態群集
    第二層 Engle-Granger Cointegration
    """
    labels = dynamic_hdbscan_clustering(features_df, sector_map)
    unique_clusters = set(labels) - {-1}
    candidates = []

    for cid in unique_clusters:
        cluster_tickers = labels[labels == cid].index.tolist()
        if len(cluster_tickers) >= 2:
            s_map = features_df.loc[cluster_tickers[0], 'sector'] if 'sector' in features_df.columns else None
            for a, b in combinations(cluster_tickers, 2):
                candidates.append((a, b, s_map))

    if not candidates:
        return {n: [] for n in top_ns}

    min_len = len(price_window) * 0.8
    norm = price_window / price_window.iloc[0]

    coint_results = Parallel(n_jobs=-1, batch_size='auto')(
        delayed(test_single_coint)(a, b, sector, price_window, coint_pval, min_len, norm)
        for a, b, sector in candidates
    )

    passed = [res for res in coint_results if res is not None]
    if not passed:
        return {n: [] for n in top_ns}

```

```

for p in passed:
    try:
        # [修正點] 依照文獻對齊: Spread = Price_A - Beta * Price_B
        beta = p['beta']
        a_norm = norm[p['stock_a']]
        b_norm = norm[p['stock_b']]

        # 計算共整價差
        spread = a_norm - beta * b_norm

        # SSD 為價差的平方和
        p['ssd'] = (spread**2).sum()

        # 計算過零率 (均值回歸特徵)
        centered = spread - spread.mean()
        zero_crossings = ((centered.shift(1) * centered) < 0).sum()
        p['zero_cross'] = zero_crossings
    except:
        p['ssd'] = np.inf
        p['zero_cross'] = 0

# 依照 SSD 排序並選出 Top-N
df_passed = pd.DataFrame(passed)
if df_passed.empty:
    return {n: [] for n in top_ns}

df_passed = df_passed.sort_values('ssd')
return {n: df_passed.head(n).to_dict('records') for n in top_ns}

```

階段三-2：交易期執行與回測主迴圈 (Backtesting Engine)

```

In [10]: def run_backtesting(price_pivot, prices_raw, vix_features, sector_map,
                             formation_window=FORMATION_WINDOW,
                             trading_window=TRADING_WINDOW,
                             rolling_window=ROLLING_WINDOW,
                             top_ns=TOP_N_PAIRS,
                             initial_cap=INITIAL_CAPITAL,
                             tranches=CAPITAL_TRANCHES):
    if isinstance(top_ns, int):
        top_ns = [top_ns]

    dates = price_pivot.index
    N = len(dates)
    all_pnl = {n: pd.DataFrame(index=dates) for n in top_ns}
    records = []
    si, wid = 0, 0

    # 每個視窗被分配到的絕對資金
    tranche_capital = initial_cap / tranches

    print(f'Starting backtest... Initial Capital: ${initial_cap}, Allocated per

    while si + formation_window + trading_window <= N:
        fe = si + formation_window
        te = fe + trading_window

        fp = price_pivot.iloc[si:fe]
        tp = price_pivot.iloc[fe:te]

```



```

# 1. 擷取這個月滾動區間內的 History Data Panel
form_start_date = dates[si]
form_end_date = dates[fe-1]

# 使用 IndexSlice 過濾 MultiIndex [date, ticker] Panel
idx = pd.IndexSlice
try:
    fp_raw = prices_raw.loc[idx[form_start_date:form_end_date, :], :]
    if 'VIX' in vix_features.columns:
        fp_vix = vix_features.loc[form_start_date:form_end_date, 'VIX']
    else:
        fp_vix = vix_features.loc[form_start_date:form_end_date]

# 2. 利用前置建立的 Pipeline, 計算此區間的截面特徵
std_features, _ = feature_engineering_pipeline(fp_raw, fp_vix)

# 3. 套用 HDBSCAN 雙層動態選股邏輯
sel = select_pairs_with_hdbscan(fp, std_features, sector_map, top_ns)
except Exception as e:
    # print(f"Window {wid} Feature Pipeline failed: {e}. Using fallback
    sel = select_pairs_for_window(fp, sector_map, top_ns)

rec = {'window_id': wid, 'form_start': dates[si], 'form_end': dates[fe-1],
       'trade_start': dates[fe], 'trade_end': dates[te-1], 'pairs_by_n':
records.append(rec)

# 用於終端機印出的追蹤字串
track_str = f"Window {wid:02d} | Trade: {dates[fe].date()} ~ {dates[te-1].date()}"

for n in top_ns:
    pairs = sel.get(n, [])
    col = f'W{wid:02d}'
    wpnl = pd.Series(0.0, index=dates)

    if pairs:
        # 將該視窗的資金均分給 Top-N 的每個配對
        pair_capital = tranche_capital / len(pairs)

        for p in pairs:
            # 取得單一配對的百分比報酬
            pct_return = execute_pair_trades(tp, p, fp)
            # 轉換為絕對金額 (百分比 * 分配資金)
            wpnl = wpnl.add(pct_return * pair_capital)

    all_pnl[n][col] = wpnl

# 針對特定 N (例如 Top-5) 印出該視窗結束時的資金變動與損益百分比
if n == top_ns[0]: # 預設印出第一個 Top-N 組合的狀況
    window_abs_pnl = wpnl.sum()
    window_pct_return = (window_abs_pnl / tranche_capital) * 100
    track_str += f" | Top-{n} PnL: ${window_abs_pnl:+.2f} ({window_pct_return:+.2f}%)

print(track_str)
si += rolling_window
wid += 1

print(f'Backtest done: {wid} windows.')
return all_pnl, records

```

```
# 自動轉換單一數值為 List 以防止後續 Iterable 報錯
if isinstance(TOP_N_PAIRS, int):
    TOP_N_PAIRS = [TOP_N_PAIRS]
```

階段三-3：啟動歷史滾動回測

```
In [11]: # 啟動回測並將結果賦值給全域變數 all_pnl 與 window_records
all_pnl, window_records = run_backtesting(
    price_pivot,
    prices_raw,
    vix_features,
    sector_map,
    formation_window=FORMATION_WINDOW,
    trading_window=TRADING_WINDOW,
    rolling_window=ROLLING_WINDOW,
    top_ns=TOP_N_PAIRS,
    initial_cap=INITIAL_CAPITAL,
    tranches=CAPITAL_TRANCHES
)

# 自動轉換單一數值為 List 以防止後續 Iterable 報錯
if isinstance(TOP_N_PAIRS, int):
    TOP_N_PAIRS = [TOP_N_PAIRS]
```

```
Starting backtest... Initial Capital: $7000, Allocated per Window: $1000.00
Window 00 | Trade: 2020-01-02 ~ 2020-07-01 | Top-1 PnL: $-111.64 (-11.16%)
Window 01 | Trade: 2020-01-31 ~ 2020-07-30 | Top-1 PnL: $+8.25 (+0.82%)
Window 02 | Trade: 2020-03-02 ~ 2020-08-27 | Top-1 PnL: $-43.55 (-4.35%)
Window 03 | Trade: 2020-03-30 ~ 2020-09-25 | Top-1 PnL: $+509.87 (+50.99%)
Window 04 | Trade: 2020-04-28 ~ 2020-10-23 | Top-1 PnL: $-135.30 (-13.53%)
Window 05 | Trade: 2020-05-27 ~ 2020-11-20 | Top-1 PnL: $+0.00 (+0.00%)
Window 06 | Trade: 2020-06-24 ~ 2020-12-21 | Top-1 PnL: $-61.01 (-6.10%)
Window 07 | Trade: 2020-07-23 ~ 2021-01-21 | Top-1 PnL: $-50.47 (-5.05%)
Window 08 | Trade: 2020-08-20 ~ 2021-02-19 | Top-1 PnL: $-58.63 (-5.86%)
Window 09 | Trade: 2020-09-18 ~ 2021-03-19 | Top-1 PnL: $-55.56 (-5.56%)
Window 10 | Trade: 2020-10-16 ~ 2021-04-19 | Top-1 PnL: $-67.38 (-6.74%)
Window 11 | Trade: 2020-11-13 ~ 2021-05-17 | Top-1 PnL: $-59.04 (-5.90%)
Window 12 | Trade: 2020-12-14 ~ 2021-06-15 | Top-1 PnL: $-49.59 (-4.96%)
Window 13 | Trade: 2021-01-13 ~ 2021-07-14 | Top-1 PnL: $-28.84 (-2.88%)
Window 14 | Trade: 2021-02-11 ~ 2021-08-11 | Top-1 PnL: $-67.41 (-6.74%)
Window 15 | Trade: 2021-03-12 ~ 2021-09-09 | Top-1 PnL: $-108.46 (-10.85%)
Window 16 | Trade: 2021-04-12 ~ 2021-10-07 | Top-1 PnL: $-119.14 (-11.91%)
Window 17 | Trade: 2021-05-10 ~ 2021-11-04 | Top-1 PnL: $-84.45 (-8.45%)
Window 18 | Trade: 2021-06-08 ~ 2021-12-03 | Top-1 PnL: $+0.00 (+0.00%)
Window 19 | Trade: 2021-07-07 ~ 2022-01-03 | Top-1 PnL: $+61.98 (+6.20%)
Window 20 | Trade: 2021-08-04 ~ 2022-02-01 | Top-1 PnL: $+0.00 (+0.00%)
Window 21 | Trade: 2021-09-01 ~ 2022-03-02 | Top-1 PnL: $+0.00 (+0.00%)
Window 22 | Trade: 2021-09-30 ~ 2022-03-30 | Top-1 PnL: $+0.00 (+0.00%)
Window 23 | Trade: 2021-10-28 ~ 2022-04-28 | Top-1 PnL: $+16.47 (+1.65%)
Window 24 | Trade: 2021-11-26 ~ 2022-05-26 | Top-1 PnL: $-102.12 (-10.21%)
Window 25 | Trade: 2021-12-27 ~ 2022-06-27 | Top-1 PnL: $+103.45 (+10.35%)
Window 26 | Trade: 2022-01-25 ~ 2022-07-26 | Top-1 PnL: $-69.76 (-6.98%)
Window 27 | Trade: 2022-02-23 ~ 2022-08-23 | Top-1 PnL: $-56.21 (-5.62%)
Window 28 | Trade: 2022-03-23 ~ 2022-09-21 | Top-1 PnL: $-45.58 (-4.56%)
Window 29 | Trade: 2022-04-21 ~ 2022-10-19 | Top-1 PnL: $-61.83 (-6.18%)
Window 30 | Trade: 2022-05-19 ~ 2022-11-16 | Top-1 PnL: $-50.95 (-5.09%)
Window 31 | Trade: 2022-06-17 ~ 2022-12-15 | Top-1 PnL: $-70.89 (-7.09%)
Backtest done: 32 windows.
```

階段四：績效評估與對比分析

指標	公式
CAGR	$(1 + r_{total})^{252/T} - 1$
Sharpe	$\frac{\mu_{ann}}{\sigma_{ann}}$
MDD	$\frac{\max(1 - NAV_t / peak_t)}{}$
Sortino	使用下行標準差

```
In [12]: def compute_metrics(ret, label=''):
    """Annualised performance metrics from daily return series."""
    r = ret.dropna().replace([np.inf, -np.inf], np.nan).dropna()
    T = len(r)
    if T == 0: return {}
    cum = (1+r).cumprod()
    tot = cum.iloc[-1]-1
    cagr = (1+tot)**(252/T)-1
    vol = r.std()*np.sqrt(252)
    sharpe = (r.mean()*252)/vol if vol>0 else np.nan
    mdd = ((cum-cum.cummax())/cum.cummax()).min()
    dn = r[r<0].std()*np.sqrt(252)
    sortino = (r.mean()*252)/dn if dn>0 else np.nan
    return {'策略':label, 'CAGR(%)':round(cagr*100,2),
            '年化波動(%)':round(vol*100,2),
            'Sharpe':round(sharpe,3) if not np.isnan(sharpe) else np.nan,
            'Sortino':round(sortino,3) if not np.isnan(sortino) else np.nan,
            '最大回撤(%)':round(mdd*100,2),
            '勝率(%)':round((r>0).mean()*100,2),
            '總報酬(%)':round(tot*100,2)}

    strat_ret = {}
    for n in (TOP_N_PAIRS if isinstance(TOP_N_PAIRS, list) else [TOP_N_PAIRS]):
        df = all_pnl[n]
        if df.empty: continue

        # 將所有平行視窗的「絕對金額損益」加總
        portfolio_daily_pnl = df.sum(axis=1)

        # 除以總初始資金，得到嚴格遵守資金控管的投資組合每日報酬率
        strat_ret[f'Top-{n}'] = portfolio_daily_pnl / INITIAL_CAPITAL

    # strat_ret['S&P500(EW)'] = price_pivot.pct_change().mean(axis=1)
    pct_change = price_pivot.pct_change()
    # 將無限大替換為 NaN
    pct_change = pct_change.replace([np.inf, -np.inf], np.nan)
    # 排除單日暴漲超過 100% 的極端錯誤數據 (通常是除權息未還原導致)
    pct_change = pct_change.where(pct_change < 1.0, np.nan)

    strat_ret['S&P500(EW)'] = pct_change.mean(axis=1)
```

```
metrics_df = pd.DataFrame(
    [m for lbl, r in strat_ret.items() for m in [compute_metrics(r, lbl)] if m]
).set_index('策略')
display(metrics_df.style.background_gradient(cmap='RdYlGn', axis=0))
```

	CAGR(%)	年化波動 (%)	Sharpe	Sortino	最大回撤 (%)	勝率(%)	總報
策略							
Top-1	-3.110000	4.420000	-0.693000	-0.664000	-14.090000	24.010000	-11.8
Top-3	1.310000	10.220000	0.178000	0.219000	-17.410000	34.030000	5.3
S&P500(EW)	18.790000	25.220000	0.810000	0.980000	-40.120000	54.920000	99.0

累積 NAV 曲線與回撤圖

```
In [13]: COLORS = px.colors.qualitative.Plotly
fig_p = make_subplots(rows=2, cols=1, shared_xaxes=True,
    row_heights=[0.65, 0.35],
    subplot_titles=['累積淨值 (NAV) 曲線', '歷史回撤 (Drawdown%)'],
    vertical_spacing=0.08)
for i, (lbl, ret) in enumerate(strat_ret.items()):
    r=ret.dropna(); nav=(1+r).cumprod()
    dd=((nav-nav.cummax())/nav.cummax()*100
    c=COLORS[i%len(COLORS)]
    fig_p.add_trace(go.Scatter(x=nav.index, y=nav.values, name=lbl,
        line=dict(color=c, width=2),
        hovertemplate=f'{{x|Y-%m-%d}}<br>NAV:{{y:.4f}}<extra>{lbl}</extra>'),
    fig_p.add_trace(go.Scatter(x=dd.index, y=dd.values, name=lbl, showlegend=False,
        fill='tozeroy', line=dict(color=c, width=1),
        hovertemplate=f'{{x|Y-%m-%d}}<br>DD:{{y:.2f}}%<extra>{lbl}</extra>'),
    fig_p.update_layout(title='配對交易策略績效對比', height=650, template='plotly_dark',
        hovermode='x unified', legend=dict(orientation='h', y=1.02, x=1, xanchor='right')
    fig_p.show()
```

匯出歷史回測與交易明細報表 (Export Reports)

本區塊定義了報表匯出功能，可將二十年回測期間每個「月滾動視窗」抓出的 HDBSCAN 配對陣列、損益軌跡與交易紀錄，轉換為 CSV 或 Excel 檔案，供給外部風控查驗與歸因分析。

```
In [17]: import os
import pandas as pd
import numpy as np
from collections import Counter

def export_performance_reports(all_pnl, window_records, top_n=5,
    initial_cap=INITIAL_CAPITAL, tranches=CAPITAL_TRA
    output_dir=r'..\results'):
    """將回測結果匯出為具備詳細統計指標、配對熱度與產業別的 CSV 報表"""

    # [修正 1] 確保底下的 window 與 year 子資料夾都確實建立，避免存檔報錯
    win_dir = os.path.join(output_dir, 'window')
    year_dir = os.path.join(output_dir, 'year')
```

```

os.makedirs(win_dir, exist_ok=True)
os.makedirs(year_dir, exist_ok=True)

allocated_cap = initial_cap / tranches

# =====
# 1. 視窗層級報表 (Window-Level Report)
# =====
win_data = []
for rec in window_records:
    wid = rec['window_id']
    ts = rec['trade_start']
    te = rec['trade_end']
    pairs = rec['pairs_by_n'].get(top_n, [])

    pair_details = [f"{p['stock_a']}-{p['stock_b']}" for p in pairs]

    sectors_in_win = list(set([p.get('sector', 'Unknown') for p in pairs]))
    sector_str = ", ".join(sectors_in_win) if sectors_in_win else "無"

    col = f'W{wid:02d}'
    wpn1 = all_pnl[top_n][col]

    trade_days = (te - ts).days
    if trade_days <= 0: trade_days = 1

    win_pnl = wpn1.sum()
    win_ret = win_pnl / allocated_cap

    active_pnl = wpn1[(wpnl.index >= ts) & (wpnl.index <= te)]
    daily_ret = active_pnl / allocated_cap

    cagr = (1 + win_ret) ** (365 / trade_days) - 1 if win_ret > -1 else -1
    std = daily_ret.std()
    sharpe = (daily_ret.mean() * 252) / (std * np.sqrt(252)) if std > 1e-8 else 0

    win_data.append({
        '視窗代碼': f'W{wid:02d}',
        '交易起始日': ts.strftime('%Y-%m-%d'),
        '交易結束日': te.strftime('%Y-%m-%d'),
        '分配資金(USD)': round(allocated_cap, 2),
        '視窗總損益(USD)': round(win_pnl, 2),
        '視窗報酬率(%)': round(win_ret * 100, 2),
        '年化報酬率(CAGR %)': round(cagr * 100, 2),
        '夏普率(Sharpe)': round(sharpe, 3),
        '涵蓋產業': sector_str,
        '入選配對數量': len(pairs),
        '配對清單 (含產業)': ", ".join(pair_details)
    })

df_window = pd.DataFrame(win_data)
# [修正 2] 統一使用 os.path.join 串接子資料夾路徑
win_csv_path = os.path.join(win_dir, f'window_report_Top{top_n}.csv')
df_window.to_csv(win_csv_path, index=False, encoding='utf-8-sig')

# =====
# 2. 年度層級報表與配對/產業熱度 (Yearly Report)
# =====
portfolio_daily_pnl = all_pnl[top_n].sum(axis=1)

```

```

yearly_data = []
years = portfolio_daily_pnl.index.year.unique()

rolling_capital = initial_cap

for y in years:
    y_pnl_series = portfolio_daily_pnl[portfolio_daily_pnl.index.year == y]
    y_pnl = y_pnl_series.sum()

    start_cap = rolling_capital
    end_cap = start_cap + y_pnl

    y_ret = y_pnl / start_cap if start_cap > 0 else 0

    y_ret_series = y_pnl_series / start_cap if start_cap > 0 else y_pnl_seri

    std = y_ret_series.std()
    sharpe = (y_ret_series.mean() * 252) / (std * np.sqrt(252)) if std > 1e-

    cum = (1 + y_ret_series).cumprod()
    peak = cum.cummax()
    dd = (cum - peak) / peak
    mdd = dd.min()

    cumulative_ret = (end_cap - initial_cap) / initial_cap

    y_pairs = []
    y_sectors = []
    for rec in window_records:
        ts, te = rec['trade_start'], rec['trade_end']
        if ts.year == y or te.year == y:
            pairs = rec['pairs_by_n'].get(top_n, [])
            y_pairs.extend([f"{p['stock_a']}-{p['stock_b']}" for p in pairs])
            y_sectors.extend([p.get('sector', 'Unknown') for p in pairs])

    heat_count = Counter(y_pairs).most_common(3)
    heat_str = ", ".join([f"{k} ({v}次)" for k, v in heat_count]) if heat_co

    sector_count = Counter(y_sectors).most_common(3)
    sector_str = ", ".join([f"{k} ({v}次)" for k, v in sector_count]) if sec

    yearly_data.append({
        '年度': y,
        '期初資金(USD)': round(start_cap, 2),
        '年度總損益(USD)': round(y_pnl, 2),
        '期末資金(USD)': round(end_cap, 2),
        '年度報酬率(%)': round(y_ret * 100, 2),
        '累積總報酬率(%)': round(cumulative_ret * 100, 2),
        '年度夏普率': round(sharpe, 3),
        '最大回撤(MDD %)': round(mdd * 100, 2),
        '年度熱門產業 (Top 3)': sector_str,
        '年度配對熱度 (Top 3)': heat_str
    })

rolling_capital = end_cap

df_yearly = pd.DataFrame(yearly_data)
# [修正 2] 統一使用 os.path.join 串接子資料夾路徑
yearly_csv_path = os.path.join(year_dir, f'yearly_report_Top{top_n}.csv')
df_yearly.to_csv(yearly_csv_path, index=False, encoding='utf-8-sig')

```

```

print(f" Top-{{top_n}} 報表輸出成功!")
print(f"1. 視窗層級報表: {{win_csv_path}}")
print(f"2. 年度層級報表: {{yearly_csv_path}}")

return df_window, df_yearly

# =====
# [修正 3] 執行外部迴圈・處理所有 N 值
# =====
# 確保 TOP_N_PAIRS 是一個列表
top_n_list = TOP_N_PAIRS if isinstance(TOP_N_PAIRS, list) else [TOP_N_PAIRS]

# 建立字典來存放產出的 DataFrame・方便後續在 Notebook 取用
all_reports = {}

for n in top_n_list:
    print(f"\n⌚ 正在產生 Top-{{n}} 的報表...")
    df_win, df_year = export_performance_reports(all_pnl, window_records, top_n=

    all_reports[n] = {'window': df_win, 'year': df_year}

    # 依序在 Jupyter 畫面上預覽年度報表
    print(f"\n=== Top-{{n}} 年度報表預覽 ===")
    display(df_year)

```

⌚ 正在產生 Top-1 的報表...

 Top-1 報表輸出成功!

1. 視窗層級報表: ..\results\window>window_report_Top1.csv

2. 年度層級報表: ..\results\year\yearly_report_Top1.csv

=== Top-1 年度報表預覽 ===

	年度	期初資金 (USD)	年度總損 益(USD)	期末資金 (USD)	年度報 酬率 (%)	累積總報 酬率(%)	年度夏 普率	最大回 撤 (MDD %)	年度熱 業 (To
0	2019	7000.0000	0.0000	7000.0000	0.0000	0.0000	0.0000	0.0000	
1	2020	7000.0000	-174.0400	6825.9600	-2.4900	-2.4900	-0.4260	-4.5500	Informa Techno (1
2	2021	6825.9600	-464.9100	6361.0500	-6.8100	-9.1300	-1.6440	-6.9400	Informa Techno (1
3	2022	6361.0500	-218.8400	6142.2200	-3.4400	-12.2500	-0.5910	-7.0400	Informa Techno (1

正在產生 Top-3 的報表...

Top-3 報表輸出成功！

- 1. 視窗層級報表: ..\results>window>window_report_Top3.csv
- 2. 年度層級報表: ..\results\year\yearly_report_Top3.csv

=== Top-3 年度報表預覽 ===

	年度	期初資金 (USD)	年度總損 益(USD)	期末資金 (USD)	年度報 酬率(%)	累積總報 酬率(%)	年度夏 普率	最大回撤 (MDD %)	年度業 (
0	2019	7000.0000	0.0000	7000.0000	0.0000	0.0000	0.0000	0.0000	
1	2020	7000.0000	-675.1200	6324.8800	-9.6400	-9.6400	-2.3350	-9.6900	Inform Techr
2	2021	6324.8800	-279.1200	6045.7500	-4.4100	-13.6300	-0.6680	-6.2300	Inform Techr
3	2022	6045.7500	1462.4800	7508.2400	24.1900	7.2600	1.0970	-19.9300	Inform Techr